

Flutter Practical Assignment

Role level: Flutter Developer

Before you start — please read this part

We are not measuring hours, and we are not looking for a feature-complete product. We want to see how you structure an app that talks to a network, and how it behaves when things go wrong.

- **Finishing the mandatory scope is a complete, successful submission.** Bonuses are genuinely optional and cannot lower your score.
- **Where the brief is ambiguous, decide and document it** in your README under "Assumptions."
- **Questions are welcome and never count against you.** Email the address in your invitation; we reply within a few working hours, Monday to Friday. If you write to us over a weekend and are blocked, take an extension — see below.
- **AI assistants are allowed.** Note in the README which tools you used and for what. The one firm rule: you will be asked to explain your submitted code line by line, so submit only what you understand.
- **Your code stays yours.** We use it only to assess this application, we will not use any of it in our products, and we delete your screen recording once a decision is made.

If you run out of time

Cut from the bottom of this list, and say in the README what you cut:

1. Tests (*keep at least two — see the note below*)
2. The offline banner's "how long ago" detail (*keep serving cached data*)
3. Pull to refresh
4. Filter or sort niceties

Do **not** cut: the list with pagination, search, the detail screen, favourites persistence, or error handling. Those are the parts we are actually reading.

The task: Explorer

Build a small app that browses a public list-based API, lets the user search it, view details, save favourites locally, and keeps working when the network drops.

Your data source

Use **The Rick and Morty API** — free, no key, no rate limit worth worrying about, and it supports everything this brief needs:

Purpose	Endpoint
Paginated list	https://rickandmortyapi.com/api/character?page=2
Search by name	https://rickandmortyapi.com/api/character?name=rick
Single item	https://rickandmortyapi.com/api/character/2

The response wraps results in **info** (with **count**, **pages**, **next**, **prev**) and **results**. Note that a search with no matches returns **404**, not an empty list — handling that correctly is part of the exercise.

Screens and requirements

1. List screen

- Paginated list. **Infinite scroll**: load the next page as the user approaches the bottom, with a loading indicator at the list's end while fetching.
- **Search field** with **debounce** (roughly 300–500 ms). Typing quickly must not fire a request per keystroke.
- **Pull to refresh**, which resets to page one.
- Each row shows at least two meaningful fields plus a favourite toggle.
- Tapping a row opens the detail screen.
- Distinct, non-overlapping UI for every one of these situations:
 - initial loading
 - loaded with results
 - loaded with **zero** results for the current search (a message naming the query, not a blank screen)
 - error (with a **retry** action)

2. Detail screen

- Show a richer view of the selected item.
- Favourite toggle, consistent with the list screen toggling in one place must be reflected in the other.
- If the item is in the local cache, it must open and display **while completely offline**.

3. Favourites screen

- Separate tab or route listing all favourited items.
- **Persisted locally** and fully functional with no network connection.
- Removing a favourite here updates the list screen too.
- Its own empty state.

4. Offline behaviour

This is a core requirement, not a bonus.

- Cache the fetched list data locally.
- When the network is unavailable, serve cached data instead of showing an error.
- Show a visible, non-blocking indicator that the data is cached and when it was fetched — for example a banner reading **Offline – showing data from 12 minutes ago**.
- Recover automatically or via retry when connectivity returns.

Technical requirements

These are mandatory.

1. **Layered structure.** Separate data, domain, and presentation concerns. Widgets must not call HTTP clients or storage directly.
2. **A repository** sits between the UI and your data sources (network + cache). The UI should not know where data came from.
3. **API DTOs are distinct from the models your UI uses**, with explicit mapping between them. A change to the API response shape should not ripple into your widgets.
4. **Typed errors.** Map network and parsing exceptions into your own failure type. Never render a raw `Exception.toString()` to the user. No empty `catch` blocks.
5. **Correct async handling:**
 - Debounced search cancels the pending timer and **ignores responses for a query the user has already changed**.
 - Pagination does not fire duplicate concurrent requests for the same page, and knows when it has reached the last page.
 - Guard against using `BuildContext` or calling `setState` after an `await` when the widget is gone.
 - Dispose controllers, timers, and subscriptions.
7. **State management:** Riverpod, Bloc/Cubit, or Provider. Your choice justify it briefly in the README.
8. **Tests:**
 - A repository unit test with a **mocked** HTTP client covering a success path and at least one failure path.
 - A test for your debounce or pagination logic.
 - One widget test exercising a real interaction.
9. **flutter analyze passes with no warnings.** Do not add `// ignore:` comments to silence them.
10. No secrets or keys committed to the repository.
11. **State which platform you built and tested on** (Android, iOS, or both) in the README. We do not expect you to own both, and testing on one is completely fine.

Optional bonus: pick at most one

Only if the mandatory scope is finished and solid. Attempting a bonus **cannot** lower your score, and skipping all of them costs you nothing.

The lighter options

- Filter chips or sort options combined correctly with the active search.
- Light/dark theme with a persisted preference.

The hard one — episodes without melting the network

This is deliberately the hardest thing in this brief, and it is genuinely optional. **There is a no-code path — see Path 2.**

The feature. On the character detail screen, show the episodes that character appears in name, air date, and episode code, grouped by season.

Why it is hard. A character object gives you **episode** as a list of URLs, not names:

```
"episode": [  
  "https://rickandmortyapi.com/api/episode/1",  
  "https://rickandmortyapi.com/api/episode/2",  
  ...  
]
```

Main characters appear in more than fifty episodes. The obvious implementation fires fifty-one requests to render one screen. That is the classic N+1 fan-out, and it is the thing we actually want to see you handle.

Before you write anything, go and find out two things about the API:

- What does `/api/episode/1,2,3` return, and how does that differ from `/api/episode/1`? The shape is not the same, and a parser written for one will break on the other. Finding this yourself is part of the exercise.
- Is there a practical limit on how many ids you can put in one URL?

What we are looking for

- **Batch** the lookups instead of requesting episodes one at a time, and chunk them at a size you can justify.
- **Parse both response shapes** correctly, including the single-id case.
- **Cache resolved episodes.** Opening a second character who shares forty episodes with the first should cause far fewer requests — ideally none.
- **Single-flight:** if two parts of the app ask for episode 12 at the same moment, exactly one network call should happen and both callers should get the result.
- **Cap concurrency.** Do not open twelve sockets at once because you happened to have twelve chunks.

- **Survive partial failure.** One chunk fails and the rest succeed — the user must not lose the episodes that did arrive, and needs a way to retry just the missing part.
- **Cancel cleanly.** If the user taps back mid-fan-out, in-flight work must not write to dead state or keep running for nothing.
- Add a small **debug overlay** showing how many network requests the screen made. If we cannot see your batching working in thirty seconds, we cannot credit it.
- Keep a **fake implementation** of the episode source so your tests never touch the real API

Path 1 — build it. Implement as much as you can. Partial is fine; say in the README what you did and did not get to.

Path 2 — design it instead. Write `docs/episode-fanout.md`, one to two pages, describing how you would build it and answering the questions below. **This path is scored the same as Path 1.** It usually takes under an hour, and a sharp design note beats a half-working implementation. If you are short on time, take this path.

Either way, we will ask you these. Have answers ready:

- A character appears in 51 episodes. How many HTTP requests does your screen make, and why that number?
- What comes back from a request for a single id versus several, and how does your parser handle both?
- The user opens a second character who shares 40 episodes with the first. How many requests now?
- Two widgets ask for episode 12 at the same instant. How many network calls happen?
- One chunk of your batch fails while the others succeed. What does the user see, and what can they do about it?
- The user taps back while five chunks are still in flight. What happens to those requests and to your state?
- How would you test all of this without hitting the real API?
- Your cache is in memory. What changes if it needs to survive an app restart?

There is no single correct set of answers. We are interested in whether you can see the edge cases at all, and whether you make a deliberate choice and can defend it. "I chunk at twenty because X, and I know it costs me Y" is exactly the kind of answer we are hoping for.

The API has no aggressive rate limit, but please be considerate a runaway loop hammering a free public service is itself a finding.

What to submit

1. **A public Git repository link** with real commit history. Please commit incrementally — a single "initial commit" containing everything tells us nothing about how you work.
2. **A README.md** containing:
 - How to run it (Flutter version, setup steps)
 - Which platform(s) you built and tested on

- **An architecture overview** — your layers, why you drew the boundaries there, and what you deliberately chose *not* to abstract because it would have been over-engineering
 - Your state management choice and reasoning
 - Your caching and offline strategy
 - **Assumptions** made where the brief was ambiguous
 - **Known limitations, bugs, and anything unfinished.** Honesty here helps you.
 - Roughly how long you spent
 - AI tools used, if any, and for what
3. **A 2–3 minute screen recording** showing the app running, including at least one offline scenario (turn on airplane mode on camera). This protects you if we hit a build problem locally. No narration needed.

Send the repo link by email to the address in your invitation.

How we will evaluate this

Full transparency here is the weighting:

What we look at	Weight
The mandatory features actually work	20%
Architecture and separation of concerns	20%
State management and async correctness	15%
Error handling and offline behaviour	15%
Tests that would actually catch a bug	15%
Code quality and readability	10%
Commit history and documentation	5%

Bonuses sit **outside** this weighting. They can raise our confidence in you; they cannot cost you anything.

What we are not testing: visual design polish, animation, custom illustrations, or feature count. A plain-looking app with clean layering and correct offline behaviour scores far higher than a beautiful one with logic scattered through its widgets.

One thing worth knowing: we will look at a test you wrote, imagine breaking the code beneath it, and ask ourselves whether the test would fail. Tests that only assert a widget renders do not count for much.

Next step

If the submission goes well, we will invite you to a **60-minute second round interview**:

- You share your screen and walk us through the code. Expect questions like *"trace what happens from tapping a row to data appearing on the detail screen"* and *"where does a network error start, and where does the user finally see it?"*
- We ask you to make one small change to the app live, while narrating your thinking. **You are not expected to finish it.** We are watching how you read unfamiliar code, how you debug, and how you ask for help when stuck. Getting stuck is normal and not held against you.
- You get time to ask us anything.

You may use your usual tools in that session, including AI assistants we will just ask you to explain what they produce.

Thank you for the time you are putting in. We know take-home assignments are a real cost, and we have tried to keep this as small as we honestly can while still being useful to both sides. If anything is unclear, please email rather than guessing in silence.